

Lecture de billet d'avion - React + Vite

Résumé technique pour capture image, barcode, OCR et pré-remplissage formulaire

Lecture billet avion - React + Vite

Oui, on repart proprement : **React + Vite uniquement**, donc app web / PWA dans le navigateur, pas Expo.

Objectif fonctionnel

L'utilisateur :

1. prend une photo du billet avec son smartphone
2. l'app lit l'image
3. l'app tente de décoder le code-barres / QR / Aztec / PDF417
4. si ça échoue, l'app fait de l'OCR sur le texte visible
5. le formulaire est pré-rempli
6. l'utilisateur vérifie et valide

Pour un billet d'avion, il faut toujours essayer **le code-barres d'abord**, car le standard boarding pass IATA BCBP utilise notamment **PDF417, Aztec, DataMatrix et QR code** pour encoder les données du billet. ([iata.org](https://www.iata.org))

Stack recommandée React + Vite

Installation

```
npm install barcode-detector tesseract.js browser-image-compression
```

Rôle de chaque outil

Besoin	Outil
Prendre une photo	<code><input type="file" capture="environment"></code>
Lire QR / Aztec / PDF417 / DataMatrix	<code>barcode-detector</code>
OCR texte du billet	<code>tesseract.js</code>
Réduire / préparer l'image	<code>browser-image-compression</code> + <code>canvas</code>
Parsing des champs	Regex TypeScript maison

`barcode-detector` est intéressant parce que c'est un polyfill/ponyfill de l'API `BarcodeDetector`, basé sur ZXing-C++ WebAssembly, et il supporte des formats utiles pour les billets comme `aztec`, `data_matrix`, `pdf417` et `qr_code`. (github.com)

`tesseract.js` est un port JavaScript de Tesseract qui peut fonctionner dans le navigateur et faire de l'OCR côté client. (tesseract.projectnaptha.com)

`browser-image-compression` sert à compresser et redimensionner les images dans le navigateur avant traitement, utile car les photos smartphone sont souvent énormes. (npmjs.com)

Architecture propre pour ton app

```
src/
  features/
    ticket-scanner/
      components/
        TicketImageInput.tsx
      services/
        compressTicketImage.ts
        scanTicketBarcode.ts
        extractTicketText.ts
        parseTicketData.ts
        readTicketFromImage.ts
      types/
        FlightTicketData.ts
```

Type de données à remplir

```
export type FlightTicketData = {
  passengerName?: string;
  departureAirportCode?: string;
  departureAirportName?: string;
  arrivalAirportCode?: string;
  arrivalAirportName?: string;
  flightNumber?: string;
  departureDate?: string;
  departureTime?: string;
  boardingTime?: string;
  gate?: string;
  seat?: string;
  bookingReference?: string;
  baggage?: string;

  source: "barcode" | "ocr" | "manual";
  rawText?: string;
  confidence?: number;
};
```

1. Capture photo dans React + Vite

```
import { readTicketFromImage } from "../services/readTicketFromImage";

type Props = {
  onTicketRead: (data: unknown) => void;
};

export function TicketImageInput({ onTicketRead }: Props) {
  async function handleChange(event: React.ChangeEvent<HTMLInputElement>) {
    const file = event.target.files?.[0];

    if (!file) return;

    const result = await readTicketFromImage(file);
    onTicketRead(result);
  }

  return (
    <input
      type="file"
      accept="image/*"
      capture="environment"
      onChange={handleChange}
    />
  );
}
```

`capture="environment"` demande au navigateur mobile d'ouvrir plutôt la caméra arrière. Ce n'est pas aussi contrôlé qu'une vraie app native, mais pour React + Vite / PWA, c'est le plus simple.

2. Compression de l'image

```
import imageCompression from "browser-image-compression";

export async function compressTicketImage(file: File): Promise<File> {
  return imageCompression(file, {
    maxSizeMB: 1.5,
    maxWidthOrHeight: 1800,
    useWebWorker: true,
  });
}
```

Pourquoi c'est important :

Photo smartphone brute : 3 à 12 Mo
OCR/barcode navigateur : lent si image trop lourde
Image compressée : traitement plus rapide et moins de mémoire

3. Lecture barcode / QR / Aztec / PDF417

```
import { BarcodeDetector } from "barcode-detector";

export async function scanTicketBarcode(file: File): Promise<string | null> {
  const bitmap = await createImageBitmap(file);

  const detector = new BarcodeDetector({
    formats: ["qr_code", "aztec", "pdf417", "data_matrix"],
  });

  const barcodes = await detector.detect(bitmap);

  if (!barcodes.length) {
    return null;
  }

  return barcodes[0].rawValue ?? null;
}
```

Sur un vrai boarding pass, si le code est reconnu, tu peux obtenir une chaîne BCBP structurée. C'est beaucoup plus fiable que l'OCR.

4. OCR fallback avec Tesseract.js

```
import { createWorker } from "tesseract.js";

export async function extractTicketText(file: File): Promise<{
  text: string;
  confidence?: number;
}> {
  const worker = await createWorker("eng");

  const result = await worker.recognize(file);

  await worker.terminate();

  return {
    text: result.data.text,
    confidence: result.data.confidence,
  };
}
```

Pour tes billets en français/anglais, tu peux tester ensuite :

```
const worker = await createWorker("fra+eng");
```

Mais pour les billets, souvent les labels sont en anglais ou en codes IATA. `eng` peut suffire au début.

5. Fonction centrale : barcode d'abord, OCR ensuite

```
import { compressTicketImage } from "../compressTicketImage";
import { scanTicketBarcode } from "../scanTicketBarcode";
import { extractTicketText } from "../extractTicketText";
import { parseTicketText } from "../parseTicketData";
import { parseBoardingPassBarcode } from "../parseBoardingPassBarcode";
import type { FlightTicketData } from "../types/FlightTicketData";

export async function readTicketFromImage(file: File): Promise<FlightTicketData> {
  const compressedFile = await compressTicketImage(file);

  const barcodeText = await scanTicketBarcode(compressedFile);

  if (barcodeText) {
    return {
      ...parseBoardingPassBarcode(barcodeText),
      source: "barcode",
      rawText: barcodeText,
    };
  }

  const ocr = await extractTicketText(compressedFile);

  return {
    ...parseTicketText(ocr.text),
    source: "ocr",
    rawText: ocr.text,
    confidence: ocr.confidence,
  };
}
```

6. Parser OCR pour ton billet test

Exemple adapté à ton billet Bordeaux → Martinique :

```
import type { FlightTicketData } from "../types/FlightTicketData";

export function parseTicketText(rawText: string): Partial<FlightTicketData> {
  const text = normalizeOcrText(rawText);

  return {
    passengerName: find(text, /PASSAGER\s+([A-ZÀ-ÿ\s/-]+)/i),
    departureAirportCode: find(text, /\b(?:DE|FROM)\s+(BOD|CDG|ORY|PTP|FDF|DSS)\b/i),
    arrivalAirportCode: find(text, /\b(?:VERS|TO)\s+(BOD|CDG|ORY|PTP|FDF|DSS)\b/i),
    flightNumber: find(text, /\bVOL\s+([A-Z0-9]{2})\s?\d{2,4}\b/i),
    departureDate: find(text, /\bDATE\s+(\d{1,2})\s+[A-Z]{3}\s+\d{4}\b/i),
    departureTime: find(text, /\bDÉPART\s+(\d{2}:\d{2})\b/i),
    boardingTime: find(text, /\bEMBARQ(?:UEMENT)?\.\.?(\d{2}:\d{2})\b/i),
    gate: find(text, /\bPORTE\s+([A-Z0-9]{1,4})\b/i),
    seat: find(text, /\bSI[ÉE]GE\s+([0-9]{1,2}[A-F])\b/i),
    bookingReference: find(text, /\bR[ÉE]SERVATION\s+([A-Z0-9]{5,8})\b/i),
    baggage: find(text, /\bBAGAGE\s+([0-9]\s?PC)\b/i),
  };
}
```

```
function normalizeOcrText(text: string): string {
  return text
    .replace(/[\|]/g, "I")
    .replace(/["']/g, '')
    .replace(/\s+/g, " ")
    .trim()
    .toUpperCase();
}

function find(text: string, regex: RegExp): string | undefined {
  return text.match(regex)?.[1]?.trim();
}
```

7. Parser barcode BCBP — version MVP

Pour le barcode, ne commence pas par une implémentation complète IATA. Fais d'abord un parser simple qui récupère les éléments les plus utiles.

```
import type { FlightTicketData } from "../types/FlightTicketData";

export function parseBoardingPassBarcode(raw: string): Partial<FlightTicketData> {
  const normalized = raw.trim();

  return {
    passengerName: extractPassengerNameFromBcbp(normalized),
    departureAirportCode: extractAirportCode(normalized, 0),
    arrivalAirportCode: extractAirportCode(normalized, 1),
    flightNumber: extractFlightNumber(normalized),
  };
}

function extractPassengerNameFromBcbp(raw: string): string | undefined {
  // Les chaînes BCBP commencent souvent par M1 + NOM/PRENOM.
  const match = raw.match(/^M\d([A-Z ]{2,30})/i);

  return match?.[1]?.replace("/", " ").trim();
}

function extractAirportCode(raw: string, index: number): string | undefined {
  const matches = raw.match(/\\b[A-Z]{3}\\b/g) ?? [];

  return matches[index];
}

function extractFlightNumber(raw: string): string | undefined {
  return raw.match(/\\b[A-Z0-9]{2}\\s?\\d{2,4}\\b/)?.[0];
}
```

Ensuite, quand tu auras de vrais exemples de billets, tu amélioreras ce parser avec des cas réels.

8. Pré-remplir ton formulaire React

```
const [form, setForm] = useState<FlightTicketData>({
  source: "manual",
});

function handleTicketRead(data: FlightTicketData) {
  setForm((current) => ({
    ...current,
    ...data,
  }));
}
```

Puis dans ton formulaire :

```
<input
  value={form.departureAirportCode ?? ""}
  onChange={(e) =>
    setForm({ ...form, departureAirportCode: e.target.value })
  }
/>
```

Important : **l'utilisateur doit toujours confirmer**, parce que l'OCR peut confondre :

```
B0D ↔ 80D
FDF ↔ F0F
12A ↔ I2A
08:05 ↔ 08:05
```

9. Mon choix technique pour ton MVP React + Vite

Version simple, gratuite, 100 % navigateur

```
npm install barcode-detector tesseract.js browser-image-compression
```

Pipeline :

```
File input caméra
↓
browser-image-compression
↓
barcode-detector
↓
si échec : tesseract.js
↓
parse regex
↓
pré-remplissage formulaire
```

↓
confirmation utilisateur

C'est la meilleure base pour avancer vite sans backend.

10. Limites à connaître

tesseract.js

Avantages :

gratuit
local
pas d'API key
pas d'envoi de données serveur
fonctionne avec React + Vite

Inconvénients :

plus lent sur smartphone bas de gamme
moins fiable si photo floue
sensible à l'angle et à la lumière
peut mal lire les accents ou codes courts

barcode-detector

Avantages :

meilleur que OCR si le billet a un vrai code-barres
compatible avec formats utiles boarding pass
peut fonctionner côté navigateur

Inconvénients :

si la photo est floue, le code ne passera pas
selon les billets, le barcode peut être masqué ou trop petit
il faudra tester avec plusieurs vrais billets

11. Pour une version plus robuste plus tard

Quand tu voudras fiabiliser :

Front React + Vite
↓
upload temporaire image vers backend AdonisJS
↓

Google Cloud Vision / AWS Textract / Azure Vision

↓

suppression image brute

↓

retour JSON au front

Mais ne mets jamais une clé Google/AWS/Azure directement dans Vite : elle serait exposée côté navigateur.

Conclusion nette

Pour ton projet actuel **React + Vite**, je recommande :

1. barcode-detector pour lire QR / Aztec / PDF417 / DataMatrix
2. tesseract.js pour OCR fallback
3. browser-image-compression pour alléger les photos smartphone
4. regex TypeScript maison pour remplir le formulaire

Le plus important : **barcode d'abord, OCR ensuite.**